
University CRUD

3rd iteration of Software Design Project

Overview

- Backend
- Frontend admin panel (multiple pages)
- CRUD for **3 entities** (using ORM)
- Input validation
- REST Api structure
- Login, Register, Password Reset (with SMS integration)

Tech Stack

- Backend: Spring Boot + JPA
- Database: PostgreSQL
- Frontend: Angular

Domain Model

Example structure (adapt to your repo):

- **Person**
 - id, name, password, age, email
- **Product**
 - **person** + registration number, graduation year
- **Professor**
 - **person** + department, academic rank
- **Enrollment**
- **Course**

-
- **Code**
 - Used to encapsulate code sent via SMS + expiration date

Special relationships

- Student, Professor **M:N** Course

Resolved through Enrollment joint table.

- Person **1:N** Code

Each person can have multiple code entities generated. Only Top 1 & most-recent code for a certain person is retrieved at one time.

Architecture

- **Controller** – REST endpoints
- **Service** – business logic + validations
- **Repository** – persistence layer, through JPA
- **Exception Handler** – global error handling

Validations performed

- Unique and proper email for users
 - Strong passwords only
-

Test Development

- Added unit tests (controller + service) & integration tests (controller)
-

API Endpoints

Person (base entity)

- `POST /persons` → create person
 - `GET /persons/{id}` → get by id
 - `GET /persons` → get all
 - `PUT /persons/{id}` → full update
 - `PATCH /persons/{id}` → partial update
 - `DELETE /persons/{id}` → delete
-

Student (extends Person)

- `POST /students` → create student
 - `GET /students/{id}` → get by id
 - `GET /students` → get all
 - `PUT /students/{id}` → full update
 - `DELETE /students/{id}` → delete
-

Professor (extends Person)

- `POST /professors` → create professor
 - `GET /professors/{id}` → get by id
 - `GET /professors` → get all
 - `PUT /professors/{id}` → full update
 - `DELETE /professors/{id}` → delete
-

Course

- `POST /courses` → create course
- `GET /courses/{id}` → get by id
- `GET /courses` → get all

-
- `PUT /courses/{id}` → full update
 - `DELETE /courses/{id}` → delete
-

Enrollment (M:N)

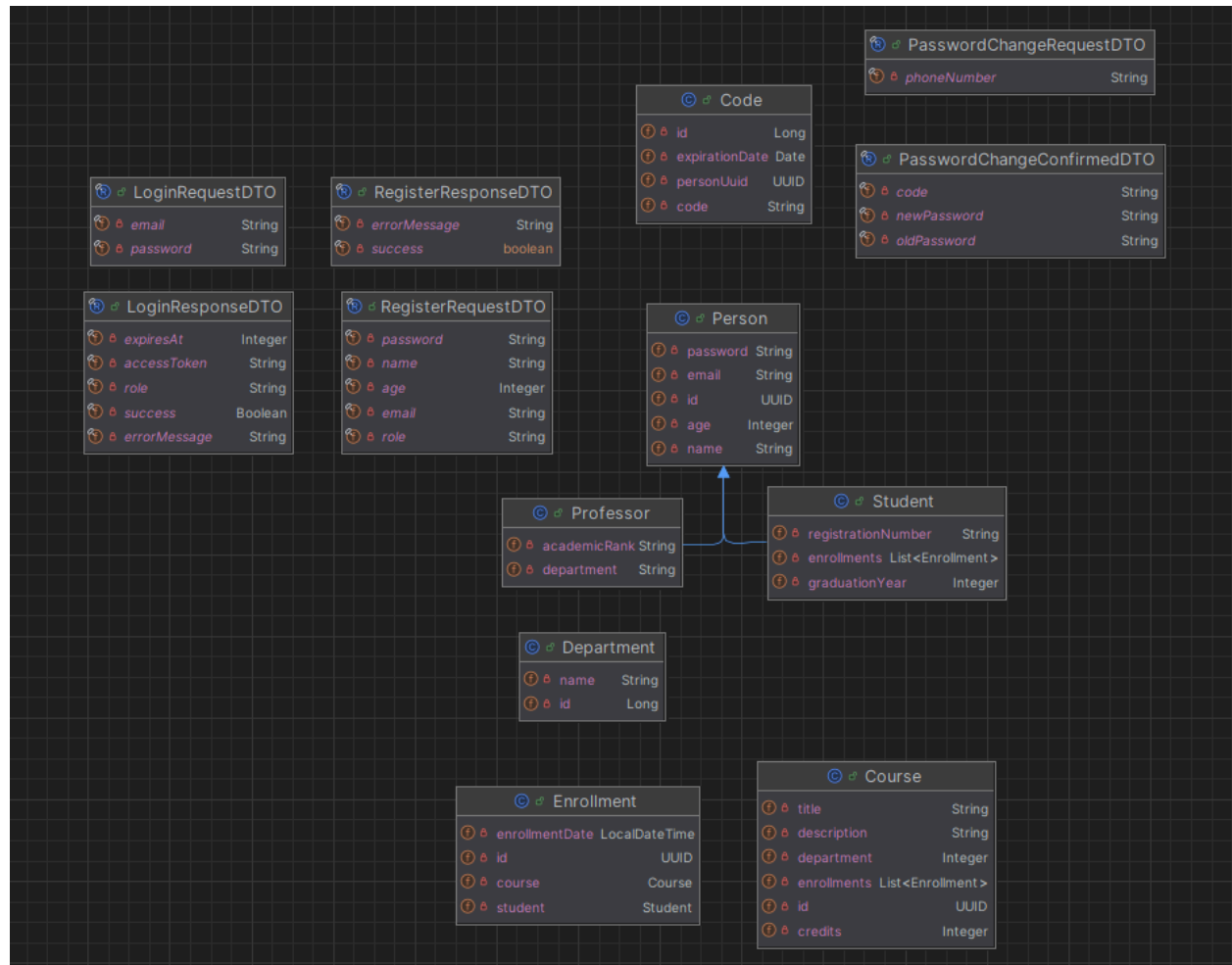
- `POST /enrollments` → enroll student in course
- `GET /enrollments` → get all enrollments
- `GET /enrollments/{id}` → get specific enrollment
- `DELETE /enrollments/{id}` → remove enrollment

Login

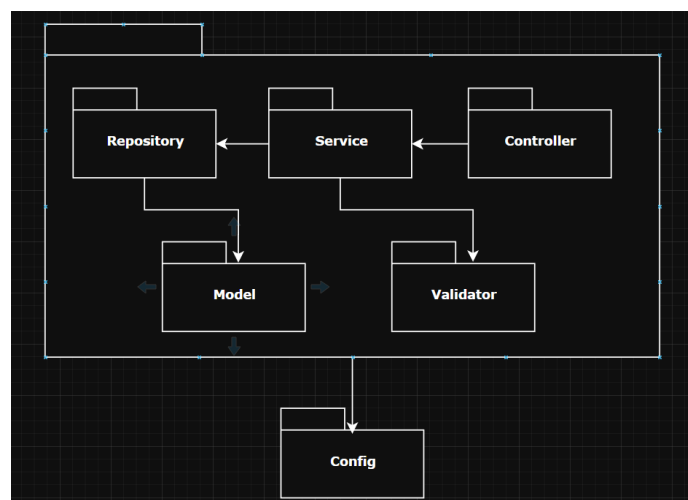
- `POST /login` → obtain login response (success / unauthorized)
- `POST /register` → obtain registering response & storing of the new user credentials (success / validation error (duplicate))

Diagrams

Class diagram:



Package diagram:



Use case Diagram:

